

GRAPH REPRESENTATIONS: A UNIFIED LIBRARY

Adjacency Matrix, Adjacency List and Incidence Matrix

DSA Assignment 1

Student Name	
Roll Number	
Programme / Branch	B.Tech CSE / CSE-AI
Institution	Amrita Vishwa Vidyapeetham
Semester	III
Course	Data Structures and Algorithms
Academic Year	2026–27

Verified status: 23 automated tests passed. Final numerical benchmark values must be inserted from the local benchmark execution before submission.

ABSTRACT

This project develops a unified graph library supporting three standard graph representations: adjacency list, adjacency matrix and incidence matrix. The implementation supports both directed and undirected graphs and provides common operations including vertex insertion, edge insertion, edge existence checking, neighbour retrieval, degree calculation and edge enumeration.

The project also implements conversions between every pair of representations and graph algorithms including Breadth First Search (BFS) and connected-components discovery. Automated testing covers normal operations, directed and undirected behaviour, conversions, algorithms and edge cases. The verified test execution collected 23 tests and passed all 23 in 0.05 seconds.

Keywords: graph representation, adjacency list, adjacency matrix, incidence matrix, BFS, graph conversion, benchmarking.

1. INTRODUCTION

A graph consists of vertices and edges connecting pairs of vertices. Graphs are used to model networks such as computer networks, transportation systems, social networks and dependency structures. The representation selected for a graph affects memory consumption, operation cost and the suitability of algorithms.

This project implements three representations behind a common interface. It studies their operations, converts graphs between them while preserving the represented structure, and applies graph algorithms directly to each representation.

2. OBJECTIVES

- Implement adjacency matrix, adjacency list and incidence matrix representations.
- Support both directed and undirected graphs.
- Provide common graph operations through a consistent interface.
- Provide edge-centric operations for the incidence matrix.
- Implement conversions between every pair of representations.
- Implement BFS and connected-components traversal.
- Test normal cases, directed/undirected cases and edge cases.
- Benchmark time and memory behaviour for sparse and dense scenarios.
- Analyse representation trade-offs.

3. PROJECT STRUCTURE

```
RollNo_Assignment1/  
  src/ - graph.py, representations.py, conversions.py, algorithms.py  
  tests/ - automated pytest test files  
  benchmark/ - benchmark.py  
  evidence/ - pytest_results.txt, benchmark_results.txt  
  report/ - Assignment1_Report.pdf  
  README.md  
  AI_Use_Declaration.txt
```

graph.py contains the common Graph interface. **representations.py** contains the three concrete graph classes. **conversions.py** contains all six pairwise conversions. **algorithms.py** contains BFS and connected-components logic.

4. GRAPH REPRESENTATIONS

4.1 Adjacency List

An adjacency list stores, for every vertex, the collection of vertices directly connected to it. The implementation uses a Python dictionary whose values are sets. Sets prevent duplicate neighbours.

Strength: efficient for sparse graphs because storage is proportional to vertices and existing edges. **Limitation:** dense graphs can make neighbour collections large.

4.2 Adjacency Matrix

An adjacency matrix stores a value for every pair of vertices. A value of 1 indicates an edge and 0 indicates no edge. For undirected graphs the matrix is symmetric. A vertex-to-index dictionary is maintained to map vertices to matrix positions.

Strength: direct edge lookup. **Limitation:** $O(V^2)$ storage regardless of the number of edges.

4.3 Incidence Matrix

An incidence matrix represents the relationship between vertices and edges. Each column corresponds to an edge. For an undirected edge, the two endpoint rows contain 1. For a directed edge, the source is represented by -1 and the destination by +1.

The implementation also keeps an edge list because edges are the natural index of this representation.

5. COMMON INTERFACE

Operation	Purpose
addVertex(v)	Add a vertex if absent.
addEdge(u,v)	Add an edge while respecting directed/undirected mode.
hasEdge(u,v)	Check whether an edge exists.
neighbours(v)	Return directly reachable neighbours.
degree(v)	Return the number of neighbours under the implementation's convention.
edges()	Return graph edges.
vertex_count()	Return number of vertices.
edge_count()	Return number of edges.

6. DIRECTED AND UNDIRECTED GRAPHS

The graph constructors accept a directed flag. In an undirected graph, an edge (u,v) also creates the reverse neighbour relationship. In a directed graph, the direction is preserved. For directed graphs, degree() in the current implementation counts outgoing neighbours. The incidence matrix explicitly records direction using -1 for the source and +1 for the destination.

7. COMPLEXITY ANALYSIS

Operation	Adjacency List	Adjacency Matrix	Incidence Matrix
Storage	$O(V + E)$	$O(V^2)$	$O(VE)$
Add vertex	$O(1)$ average	$O(V)$	$O(E)$
Add edge	$O(1)$ average	$O(1)$ after indexing	$O(V)$
hasEdge	$O(1)$ average	$O(1)$	$O(E)$
neighbours	$O(\deg(v))$	$O(V)$	$O(E)$
degree	$O(1)$ stored count	$O(V)$	$O(E)$
Enumerate edges	$O(V + E)$	$O(V^2)$	$O(E)$

The incidence matrix requires one row for every vertex and one column for every edge, giving $O(VE)$ storage. This is the key scalability limitation for dense graphs.

8. GRAPH CONVERSIONS

Source	Target	Method
Adjacency List	Adjacency Matrix	Add vertices, then traverse adjacency sets and add edges.
Adjacency Matrix	Adjacency List	Add vertices, enumerate matrix edges and add them.
Adjacency List	Incidence Matrix	Add vertices and create one incidence column per edge.
Incidence Matrix	Adjacency List	Use stored edge endpoints to add edges.
Adjacency Matrix	Incidence Matrix	Enumerate matrix edges and create incidence columns.
Incidence Matrix	Adjacency Matrix	Use stored edge endpoints to populate matrix entries.

Round-trip conversion tests verify that converting a graph to another representation and back preserves its vertices and edges.

9. GRAPH ALGORITHMS

9.1 Breadth First Search (BFS)

BFS explores a graph level by level using a queue and a visited set. A vertex is inserted into the queue only when it is first discovered. The implementation uses the common `neighbours()` operation, allowing the same BFS code to operate on all three representations.

9.2 Connected Components

Connected components are obtained by starting BFS from every unvisited vertex. Each BFS produces one component, and the process continues until all vertices are visited. For directed graphs, the current implementation follows outgoing edges; it is therefore a reachability-based component routine rather than a strongly connected components algorithm.

10. TESTING AND VALIDATION

The verified pytest execution collected 23 tests and all 23 passed in 0.05 seconds. The evidence file `pytest_results.txt` was generated from the verbose test command and a second pytest run after creating the evidence file also passed.

Test coverage includes algorithms, basic graph operations, directed and undirected graphs, conversions and edge cases.

Category	Coverage
Algorithms	BFS, single vertex, one/two/three components
Basic operations	Vertex/edge insertion, lookup, neighbours, degree, edges
Directed / undirected	Direction and degree behaviour
Conversions	All six pairwise conversions and round-trip checks
Edge cases	Empty graph, isolated vertex, duplicates, missing vertex/edge, no edges
Total	23 tests passed

11. BENCHMARK METHODOLOGY

The benchmark uses `time.perf_counter()` for elapsed time and `tracemalloc` for approximate Python memory usage. Measurements are machine-dependent and should be reported using the actual final execution.

11.1 Sparse graph: 50,000 vertices with approximately average degree 4. The adjacency list and incidence matrix can be evaluated at this scale. A full 50,000 × 50,000 Python adjacency matrix is intentionally not allocated because its $O(V^2)$ storage is impractical.

11.2 Dense bipartite graph: 2,000 vertices divided into two 1,000-vertex partitions, with approximately 40% of possible cross-partition edges. Adjacency list and adjacency matrix can be evaluated at this scale. A full incidence matrix at this density is intentionally not allocated because its $V \times E$ storage becomes extremely large.

11.3 Common-size comparison: a 2,000-vertex sparse graph is used to obtain direct timings for all three representations for construction, `hasEdge`, `neighbours` and `BFS`.

11.4 Benchmark result table

Important: The numerical cells below are intentionally marked for insertion from the actual local benchmark. No benchmark number is fabricated in this report.

Representation	Construction ms	hasEdge ms	neighbours ms	BFS ms
Adjacency List	RUN BENCHMARK	RUN	RUN	RUN
Adjacency Matrix	RUN BENCHMARK	RUN	RUN	RUN
Incidence Matrix	RUN BENCHMARK	RUN	RUN	RUN

12. DISCUSSION AND ANALYSIS

The adjacency list is naturally suited to sparse graphs because it stores existing neighbours rather than every possible vertex pair. The adjacency matrix provides direct edge access after indexing but pays $O(V^2)$ storage. The incidence matrix is naturally edge-centric, but its $O(VE)$ storage can become very large when the edge count is high.

BFS also reflects these structural differences. An adjacency list can enumerate only existing neighbours, while an adjacency matrix scans a matrix row and an incidence representation must inspect edge information. Therefore there is no single representation that is optimal for every graph and every operation; graph density and workload determine the appropriate choice.

The final benchmark discussion should connect these theoretical expectations to the actual measured values recorded on the submission machine.

13. EDGE CASES AND DESIGN DECISIONS

- Duplicate vertices do not increase the vertex count.
- Duplicate edges do not increase the edge count.
- Queries for missing vertices return an empty neighbour list or degree 0.
- Queries for missing edges return `False`.
- Empty graphs and isolated vertices are supported.
- Directed and undirected behaviour is explicitly tested.

The implementation treats the graph as a simple graph with respect to duplicate edges. Self-loop behaviour should be documented explicitly if loops are included in the final requirements.

14. CONCLUSION

The project implements a unified library for adjacency list, adjacency matrix and incidence matrix representations, together with directed/undirected support, pairwise conversions, BFS, connected components and automated validation.

The recorded testing result provides strong functional evidence: 23 tests were collected and all 23 passed. The project also includes a benchmark methodology designed to measure practical performance while avoiding impossible dense allocations.

The overall analysis demonstrates the central trade-off: adjacency lists are well suited to sparse graphs, adjacency matrices provide convenient direct edge access and are more natural for dense graphs, while incidence matrices provide an edge-centric representation but can become memory-intensive as both V and E grow.

15. FUTURE IMPROVEMENTS

- Add `removeVertex()` and `removeEdge()` operations.
- Add explicit in-degree and out-degree operations.
- Implement strongly connected components for directed graphs.
- Use compact numerical storage for large matrices.
- Support weighted graphs.
- Repeat benchmarks and report averages and variation.

16. HOW TO RUN

Install pytest:

```
python -m pip install pytest
```

Run tests:

```
python -m pytest -v
```

Save test evidence:

```
python -m pytest -v > evidence\pytest_results.txt
```

Run benchmark:

```
python benchmark\benchmark.py
```

Save benchmark evidence:

```
python benchmark\benchmark.py > evidence\benchmark_results.txt
```

17. AI USE DECLARATION

AI assistance was used to understand the assignment requirements, discuss implementation approaches, identify possible edge cases and improve code organization and documentation. The implementation was reviewed and tested in the local environment. Final testing, benchmark measurements, analysis and conclusions should be verified by the student from the executed program.

APPENDIX A — VERIFIED TEST RESULT

Recorded final pytest execution: **23 passed in 0.05s**. The pytest configuration file is present and the evidence file was generated successfully. The test suite was run again after creating the evidence file and still completed with all 23 tests passing.